

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	N.KALYAN REDDY	Reg. No.:	192424232
Date:	18/08/2026	Duration:	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and	Well-presented diagram and documentation with minor issues	Adequate diagram/documentaion but lacks clarity	Poorly presented, incomplete, or difficult to understand

	documentation			
--	---------------	--	--	--

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Project Overview

The proposed system is an intelligent smart parking platform that combines AI, IoT sensors, cameras, GPS, mobile applications and cloud services to provide real-time parking information. It detects vacant spaces, guides drivers to suitable spaces, predicts parking demand, supports reservations and digital payments, and provides occupancy and utilization analytics for parking administrators.

2. CI/CD Requirements and Analysis

Because the system contains mobile, web/backend, AI and IoT-related components, the CI/CD pipeline must automatically validate every code change before it reaches users. The pipeline should reduce integration errors, maintain software quality, protect sensitive information and provide reliable deployment and rollback.

CI/CD Need	Requirement for the Smart Parking System
Frequent integration	Developers need a shared source-control workflow so backend, mobile and AI changes can be integrated safely.
Automated builds	Every accepted change should produce a reproducible application build and package.
Automated testing	Unit, API, integration and selected AI/data-validation tests should run automatically.
Quality assurance	Static analysis, code coverage and quality gates should prevent poor-quality code from moving forward.
Security	Secrets, dependencies, containers and application endpoints should be checked before deployment.
Multiple environments	Development, staging and production must be separated to reduce production risk.
Monitoring and rollback	Deployment health must be checked and a previous stable release must be available for rollback.

3. CI/CD Pipeline Architecture and Workflow

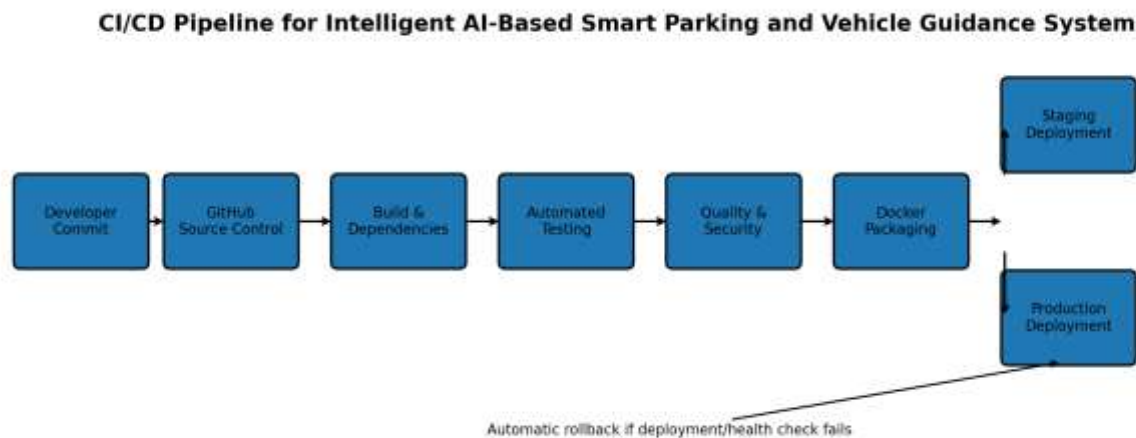


Figure 1. Proposed CI/CD workflow from code commit to production deployment.

1. Source Code Management

Developers commit application, AI, infrastructure and configuration changes to GitHub. Pull requests are used for review before merging into the main branch.

2. Build

GitHub Actions installs dependencies, validates the project structure and creates reproducible application artifacts. Docker images are built for backend services.

3. Automated Testing

Unit tests verify individual functions, API tests verify service behavior, integration tests validate communication between components, and AI/data tests check expected input formats and model behavior.

4. Code Quality and Security

SonarQube checks code quality and coverage. Dependency scanning and secret detection identify common security risks. OWASP ZAP can be used against the staging web/API deployment.

5. Packaging

A versioned Docker image and deployment artifact are created only after tests and quality gates pass.

6. Staging Deployment

The validated version is deployed to a staging environment that mirrors production. Smoke tests and API health checks are executed.

7. Production Deployment

Only approved builds are deployed to production. A rolling or blue-green strategy can reduce downtime and allow quick recovery.

8. Monitoring and Rollback

Application health, API response, parking occupancy service status and deployment logs are monitored. If a release fails health checks, the previous stable version is restored.

4. Tools and Technology Selection

Tool / Technology	Pipeline Stage	Justification
GitHub	Source code management and pull-request review	Provides version control, collaboration, branch protection and integration with GitHub Actions.
GitHub Actions	CI/CD automation	Runs build, test, security, packaging and deployment jobs from workflow files stored with the project.
Python / Node.js toolchain	Backend and service builds	Supports implementation of APIs, AI services and supporting application components.
PyTest / API test framework	Automated testing	Automates unit and service-level validation for backend and AI modules.
SonarQube	Code quality	Detects bugs, code smells and coverage problems and can enforce a quality gate.
OWASP ZAP	Web/API security testing	Performs automated security checks against the staging application.
Docker	Packaging	Creates portable, repeatable deployment images and reduces environment differences.
Container Registry	Artifact storage	Stores versioned Docker images so the exact tested build can be deployed.
Cloud Platform	Deployment and infrastructure	Provides scalable hosting, monitoring, networking and environment separation.

5. Automated Testing Strategy

Testing Type	Purpose
Unit Testing	Tests individual functions such as parking-slot status calculation, reservation validation, pricing rules and demand-prediction utilities.
API Testing	Checks parking availability APIs, reservation APIs, authentication, payment callbacks and administrative endpoints.
Integration Testing	Validates communication among the application backend, database, IoT/sensor service, AI service and notification components.
AI/Data Validation	Checks data schema, missing values, invalid sensor readings and basic model output constraints before model-related changes are accepted.
UI/Smoke Testing	Validates important user journeys such as viewing available spaces, selecting a slot and confirming a reservation.
Security Testing	Scans dependencies and staging APIs for common vulnerabilities and verifies that secrets are not committed to source control.

6. Deployment Strategy

Environment	Deployment Process
Development	Used by developers for feature development and early testing. Changes are built automatically for development branches or pull requests.
Testing / Staging	Receives only builds that pass CI quality gates. It uses production-like configuration and performs integration, smoke and security checks.
Production	Receives only approved, tested and versioned artifacts. Deployment is controlled, monitored and reversible.

A blue-green or controlled rolling deployment is recommended for production. The new version is first validated, traffic is shifted gradually, and the previous version remains available for rapid rollback.

7. Security, Quality, Notifications and Rollback

- Store database passwords, API keys, cloud credentials and payment secrets in GitHub Secrets or a managed secret store rather than source code.
- Use branch protection and pull-request approval before merging production-bound changes.
- Run dependency vulnerability scans and static code analysis on every important build.
- Use least-privilege permissions for CI/CD service accounts and deployment credentials.
- Use HTTPS/TLS for communication between mobile clients, APIs, cloud services and administrative interfaces.
- Send pipeline notifications for failed builds, failed tests, security-gate failures and production deployment results.
- Keep versioned deployment artifacts so the last known-good release can be restored.
- Use health checks, logs and monitoring after deployment; automatically or manually roll back when critical health checks fail.

8. Example CI/CD Workflow Configuration

A GitHub Actions workflow can be organized into dependent jobs so that deployment occurs only when earlier stages succeed. The following logical sequence represents the recommended configuration:

1. Trigger: pull request and push to the main branch.
2. Checkout source code.
3. Install dependencies and build the application.
4. Run unit, API and integration tests.
5. Run code-quality and security checks.
6. Build and tag the Docker image.

7. Push the approved image to the container registry.
8. Deploy the image to staging.
9. Run smoke and security checks in staging.
10. Require production approval for the release.
11. Deploy the same tested image to production.
12. Run production health checks and notify the team.
13. Rollback to the previous stable image if critical deployment checks fail.

9. Why This Pipeline Fits the Smart Parking System

The smart parking platform depends on continuously changing data from parking sensors and cameras and must provide reliable information to drivers. A CI/CD pipeline is therefore important because a faulty release could produce incorrect parking availability, reservation failures or service downtime. Automated testing, quality gates, security checks, staging validation and controlled production deployment reduce these risks. Containerized and versioned releases also make the system easier to maintain and roll back.

10. Expected Benefits

- Reduced parking search time through reliable and continuously deployed availability services.
- Lower traffic congestion and fuel consumption by guiding drivers to vacant spaces.
- Improved parking utilization through demand prediction and occupancy analytics.
- Better user convenience through reservations and digital payment support.
- Improved operational reliability through automated testing, monitoring and rollback.
- Greater security and maintainability through quality gates, secret management and controlled releases.

11. Conclusion

The proposed CI/CD pipeline provides a structured path from source-code commit to reliable deployment of the AI-based smart parking platform. GitHub and GitHub Actions manage integration and automation, while automated testing, quality analysis, security checks, container packaging, staging validation and controlled production deployment improve software reliability. Monitoring and rollback ensure that failures can be handled quickly without unnecessarily affecting parking users or administrators.

Basis of design: The pipeline structure follows the requirements in the uploaded CSA10 CO4 Assessment Tool 2 document, including source management, build, testing, quality analysis, packaging, deployment, security, notifications, rollback and a pipeline diagram.